

Part 2. 번호판 인식

Claude Code로 처음부터 만들기 — YOLO+OpenCV+OCR 파이프라인

완벽한 코드를 짜는 게 목표가 아니라, AI에게 단계별로 일을 시키고 결과를 확인하는 흐름을 체득하는 것이 목표입니다.

강사: 김생근

2026년 7월 · Part 2

Part 1의 지도를 실제로 써보는 시간입니다

Part 1에서 "검출→전처리→인식/분류→후처리" 4단계 파이프라인과, "범용 대상엔 MediaPipe, 내 도메인 대상엔 YOLO"라는 판단 기준을 배웠습니다. 번호판은 정확히 후자입니다 — 세상 어디에도 없는 우리나라 번호판 전용 인식기를, 지금부터 4단계로 직접 만듭니다.

이 실습의 진짜 목표는 완성된 코드가 아닙니다. Claude Code(또는 어떤 AI 코딩 도구든)에게 단계마다 일을 시키고, 그 결과를 직접 실행해서 확인하고, 안 되면 에러 메시지를 그대로 다시 보여주는 흐름 — 이 세 동작의 반복이 바이트 코딩의 실체입니다.

학습 목표

#	이 파트를 마치면
1	검출→전처리→인식→후처리 4단계 파이프라인을 직접 구현할 수 있다
2	AI가 처음 제안한 코드가 왜 실패하는지 원인을 스스로 진단할 수 있다
3	결정적 로직(정규식 후처리)엔 모델을 쓰지 않는다는 원칙을 적용할 수 있다
4	에러 메시지를 그대로 AI에게 다시 보여주는 디버깅 루프를 체득한다

Step 1~2. 요구사항부터, 그리고 첫 번째 함정

🤔 왜 spec.md부터 쓰나?

바이브 코딩의 시작은 코드가 아니라 요구사항 정리입니다. "한국 번호판을 인식하는 Streamlit 앱"이라고만 던지면 AI가 뭔가를 제안하는데, 그걸 그대로 받지 않고 검증하는 것이 이 단계의 핵심입니다 — AI가 제안한 아키텍처가 말이 되는지는 결국 사람이 판단해야 합니다.

💡 비유로 이해하기

설계도 없이 건물을 짓기 시작하면 3층까지 올린 뒤에야 "엘리베이터 자리가 없다"는 걸 발견합니다. spec.md는 그 설계도입니다 — 4단계(검출·전처리·인식·후처리) 구조를 코드 짜기 전에 문서로 먼저 합의합니다.

🔍 원리 설명 — Step 2. YOLO 검출의 함정

💻 코드

```
from ultralytics import YOLO

m = YOLO("yolov8n.pt")

print(m.names) # {0: 'person', ...} — "license plate" 없음!
```

AI는 십중팔구 기본 YOLO 모델을 제안합니다. 직접 실행해보면 번호판이 하나도 안 잡힙니다 — COCO 80개 클래스에 애초에 "번호판"이 없기 때문입니다. 번호판 전용 학습 모델로 교체해야 합니다.

🤖 실무 관점

여기서 핵심 설계 하나: 검출 실패 시 OpenCV 폴백으로 자동 전환하되, 어느 경로로 검출됐는지 결과에 `method` 필드로 남겨 화면에 표시합니다. 이게 없으면 "YOLO로 검출된 줄 알았는데 실제로는 계속 폴백만 타고 있었다"는 걸 못 알아챱니다 — 성공/실패보다 위험한 건 성공한 줄 착각하는 것입니다.

? 범용 YOLO 모델에 없는 클래스는 왜 없나요?

COCO 데이터셋 80개 클래스는 사람·차·동물 같은 일상 사물 위주로 구성되어 있고, 번호판처럼

도메인 특화 대상은 원래 포함되지 않습니다. Part 1에서 배운 "내 도메인 대상엔 커스텀 학습이 필요하다"는 원칙이 정확히 여기서 실감됩니다.

Step 3~4. 전처리는 순조롭게, OCR에서 두 번째 함정

Step 3. OpenCV 전처리 — 리사이즈 → 그레이스케일 → CLAHE → 이진화

🔍 원리 · 🤖 실무 관점

순수 OpenCV라 AI가 짜준 코드가 대체로 한 번에 돕니다. 다만 "조명 보정(CLAHE, 적응형 히스토그램 균등화)"을 빼먹기 쉬운데, 실제 사진은 그늘·역광이 있어 CLAHE 없이 바로 이진화하면 결과가 지저분해집니다 — Part 1에서 배운 "OpenCV는 조명이 통제된 환경에 강하다"의 반대 사례를, 이번엔 보정으로 직접 해결합니다.

Step 4. PaddleOCR 인식 — 가장 많이 막히는 단계

😞 왜 여기서 막히나?

💻 코드

```
TypeError: PaddleOCR.predict() got an unexpected  
keyword argument 'cls'
```

Claude Code는 학습 데이터 기준으로 PaddleOCR 2.x API(`ocr.ocr(img, cls=True)`)를 제안할 확률이 높습니다. 설치된 버전이 3.x면 위 에러가 납니다.

🔍 원리 설명

3.x부터는 `.ocr()` 대신 `.predict()`를 쓰고, 결과 포맷도 `[[bbox,(text,conf)],...]`가 아니라 ``result[0]["rec_texts"]` / `["rec_scores"]` / `["rec_polys"]` 딕셔너리로 바뀝니다. 이것 반영해야 합니다.`

🤖 실무 관점

AI는 학습 시점의 API 지식을 기준으로 코드를 짭니다. 라이브러리가 그 이후 버전업했으면 반드시 어긋납니다 — 이게 AI 코딩의 구조적 한계입니다. 대응은 간단합니다: 에러 메시지를 그대로 AI에게 다시 보여주면, 대부분 스스로 원인을 찾아 고칩니다.

? 버전 문제는 어떻게 미리 피하나요?

완전히 피할 수는 없습니다 — AI의 지식은 항상 어느 시점에 멈춰 있고 라이브러리는 계속 업데이트되기 때문입니다. 대신 "설치된 버전을 먼저 확인하고, 에러가 나면 그 메시지를 그대로 AI에게 다시 보여준다"는 대응 루틴을 체득하는 게 현실적입니다.

AI가 처음 준 코드가 안 돌아가는 건 실패가 아니라 디버깅 루프의 시작일 뿐입니다.

Step 5~6. 후처리는 사람이, 통합은 마지막에

Step 5. 후처리 — AI에게 시키지 않는다

🤔 왜 이번엔 AI에게 안 시키나?

한국 번호판 포맷(숫자3+한글1+숫자4, 또는 지역명+숫자+한글+숫자) 검증과 O/0·l/1 같은 OCR의 흔한 오인식 보정은 순수 로직(정규식+매핑)입니다. 입력과 출력이 결정적으로 정해진 문제에는 모델이 낄 자리가 없습니다.

💡 비유로 이해하기

계산기가 필요한 곳에 챗봇을 쓰지 않는 것과 같습니다. "이 텍스트가 번호판 포맷에 맞는가"는 규칙 하나로 100% 정확히 판정되는데, 굳이 모델에게 물어보면 속도도 느려지고 가끔 틀리기가까지 합니다.

🔍 원리 설명

이건 이 워크샵 CLAUDE.md의 Rule 5와 같은 원칙입니다: 결정적 변환(정규식 매칭, 문자 치환)에는 모델을 쓰지 않습니다. `postprocessor.py`의 `CHAR\_CORRECTIONS` 맵이 그 구현입니다.

Step 6. Streamlit 통합

4개 모듈(검출·전처리·인식·후처리)을 `run\_pipeline()` 함수 하나로 엮고, 업로드 UI를 붙입니다. 여기까지 오면 이미지를 올리는 것만으로 4단계 파이프라인 전체가 자동으로 도는 데모가 완성됩니다.

? 왜 순서가 "AI에게 시키기" → "사람이 직접"으로 갈리나요?

비결정적 판단(검출 위치, 텍스트 인식)은 AI/모델이 맡고, 결정적 변환(포맷 검증, 문자 치환)은 사람이 직접 규칙으로 짭니다 — Part 1의 "학습된 모델이 필요한 순간과 필요 없는 순간"을 가르는 기준이 여기서도 그대로 적용됩니다.

실측으로 보는 완성도

4단계를 다 이었을 때 실제로 몇 개나 맞는지, 그리고 오늘 겪을 가능성이 높은 함정 4가지를 미리 정리합니다.

실측 — 전체 검증 + 트러블슈팅 요약

증상	원인	해결
번호판이 하나도 검출 안 됨	기본 yolov8n.pt는 번호판 클래스가 없음	번호판 전용 모델로 교체
TypeError: predict() got an unexpected keyword argument 'cls'	PaddleOCR 3.x가 .ocr(cls=True) 제거	.predict() + rec_texts/rec_scores 파싱
OCR 텍스트에 O/I 같은 영문자가 섞여 나옴	이미지 품질로 숫자를 영문자로 오인식	postprocessor.py의 CHAR_CORRECTIONS로 보정
macOS에서 카메라 입력 탭이 빈 화면	카메라 권한 미승인	시스템 설정 > 개인정보 보호 > 카메라 허용
→ 실무 결론: 정답 코드 기준 8개 합성 샘플 8/8 정확도. 틀리면 어느 단계(검출/OCR/후처리)에서 인지 각 단계 출력으로 바로 특정됩니다. <i>출처: 이 워크샵 src/part2_alpr_tutorial/ (원본 TUTORIAL.md 기준, 8개 합성 샘플 실측)</i>		

시간 배분 참고

이 실습의 단계별 예상 소요시간(사전준비 5분·spec.md 15분·YOLO 30분·전처리 20분·OCR 30분·후처리 15분·Streamlit 30분·검증 10분)을 합치면 155분인데, 배정된 시간은 120분입니다.

막히는 단계(대부분 Step 4 OCR)에서 시간을 더 쓰게 되면, 후처리·통합 단계는 정답 코드를 참고해 속도를 내는 것도 방법입니다.

1~2분 요약

1~2분 요약 — 이것만 기억하세요

1. 검출(YOLO, 전용 모델 필요) → 전처리(OpenCV, CLAHE) → 인식(PaddleOCR, 버전 확인) → 후처리(정규식, 사람이 직접) 4단계.
2. AI가 처음 준 코드는 대부분 한 번에 안 됩니다 — 기본 모델에 클래스가 없거나, 라이브러리 버전이 안 맞습니다.
3. 에러 메시지를 그대로 AI에게 다시 보여주는 것이 디버깅의 시작입니다. 결정적 로직(후처리)은 AI가 아니라 사람이 직접 짭니다.

완성된 코드보다 중요한 건, 막혔을 때 원인을 찾아가는 이 흐름 자체입니다.

빠른 참조 카드

4단계 체크리스트

단계	확인할 것
① 검출	m.names에 원하는 클래스가 있는가 — 없으면 전용 모델 필요
② 전처리	조명 보정(CLAHE)을 빼먹지 않았는가
③ 인식	설치된 라이브러리 버전을 먼저 확인했는가
④ 후처리	결정적 변환에 모델을 쓰고 있지 않은가

버전 확인 명령

```
python3 -c "import paddleocr; print(paddleocr.__version__)"
```

이번 실습 위치 — 오늘 하루 안에서

시간	파트	이 파트와의 관계
09:30-10:20	Part 1. 비전 AI 지도	4단계 파이프라인 개념을 여기서 배웠음
10:30-12:30	Part 2. 번호판 인식(지금)	그 개념을 YOLO+OpenCV+OCR로 직접 구현
13:30-14:30	Part 3. MediaPipe 도전과제	"전용 모델 학습"과 반대인 "사전학습 모델 바로 쓰기"를 대비 체험